

wire-shark - Multithreaded bakery simulation

Part C — Fighting for the resources (20 Marks)

A office bakery has four ovens, four chefs, and a waiting area that can accommodate four customers on a sofa waiting for their cake to be baked and that has standing room for additional customers. As the office bakery is a new one and small scale, it is limited to accommodate 25 customers. A customer will not enter the office bakery if it is filled to capacity with other customers.

Once inside, the customer takes a seat on the sofa or stands if the sofa is filled. When a chef is free, the customer that has been on the sofa the longest is served and, if there are any standing customers, the one that has been in the shop the longest takes a seat on the sofa.

When a customer's cake is finished, any chef can accept payment, but because there is only one cash register, payment is accepted for one customer at a time. There is no separate staff as this a small office bakery. Chefs handle payment and also bake the cakes. Chefs divide their time among baking cakes, accepting payment, and learning new recipes while waiting for a customer.

In other words to make the question clear, the following constraints apply:

Customers on arriving the office bakery invoke the following functions in order: `enteroffice bakery`, `sitOnSofa`, `getcake`, `pay`.

Chefs invoke `bakecake` and `acceptPayment`. They should prioritise taking money over baking cake.

Customers cannot invoke `enterofficebakery` if the shop is at capacity.

If the sofa is full, an arriving customer cannot invoke `sitOnSofa`.

When a customer invokes `getcake` there should be a

corresponding chef executing bakecake concurrently, and vice versa.

It should be possible for up to four customers to execute get cake concurrently, and up to four chefs to execute bakecake concurrently.

The customer has to pay before the chef can acceptPayment.

The chef must acceptPayment before the customer can exit. Chefs ids are 1,2,3,4 Note: Take every action which customer performs takes 1 second. That means Customer entering to sitting in sofa takes 1 second. But chef's action baking cake and accepting payment takes 2 seconds.

Once the customer sits on sofa, the seat is reserved for him till he leaves after the payment.

Any action cannot be split. If a person initiated an action, he should continue till he is done. For example, chef should give 2 seconds for payment. He cannot split it.

Input :

<time_stamp> Customer <id>

you should accept customers till a <EOF> flag.

Example :

1 Customer 254367

2 Customer 24568

3 Customer 123456

<EOF>

Output :

<time_stamp> <Customer/Chef> <id> <actions>

Customer actions enters/sits/requests
cake/pays/leaves

Chef actions bakes/accept payments.

you don't need to print chef learning new recipes

Note: As this is a multithreading assignment, you should use threads. Each of the chef is a thread and

each of customer is another thread.

give me the necessary code to accomplish this, also make test inputs that are notorious for this ps in C. Assume that if a customer finds the bakery full, he just leaves.

Solution — Multithreaded Bakery (C, pthreads)

Below is a complete C program that implements the bakery problem using `pthread`, `sem_t` and condition variables. It respects the constraints you listed:

- 4 chefs (threads), 4 sofa seats, capacity 25 customers.
- Customers arrive at given timestamps (input lines "`<timestamp> Customer <id>`"). If bakery is full they leave immediately.
- Sofa seats are reserved until the customer finishes payment and leaves.
- Standing customers wait for a sofa seat; when a seat is freed (on a customer's leave), the standing customer who entered earliest takes it.
- When a chef is free they prioritize **accepting payment** over **baking**.
- When a chef serves a customer, chef and customer perform `bakecake` and `getcake` concurrently (chef's baking: 2 sec, customer's `getcake` action: 1 sec) and up to 4 cooks can bake concurrently.
- Customer `pay` action takes 1 second (occurs after cake is finished). After the customer finishes `pay`, they wait until a chef `acceptPayment` (2 sec) completes for them, then the customer leaves.
- Each action prints a line with the current simulated time (seconds since program start).
- Customer and chef actions are atomic (not split) with the required durations.

How to run

1. Save the code below to e.g. `bakery.c`.
2. Compile:

```
bash
```

```
gcc -pthread -o bakery bakery.c
```

3. Run and provide arrival lines (or redirect an input file). The program reads arrival lines until a line with `<EOF>` is given:

```
bash
```

```
./bakery < input.txt
```

Input lines format (exactly):

```
php-template
```

```
1 Customer 254367
```

```
2 Customer 24568
```

```
3 Customer 123456
```

```
<EOF>
```

Timestamps are seconds relative to program start (non-decreasing). The program will spawn each customer thread at the specified second.

The code

```
c
```

```
// bakery.c
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <string.h>
#include <sys/time.h>
#include <errno.h>

#define MAX_CAPACITY 25
#define SOFA_SEATS 4
#define NUM_CHEFS 4

// ----- Utilities -----
```

```

long program_start_sec() {
    static struct timeval start;
    static int initd = 0;
    if (!initd) { gettimeofday(&start, NULL); initd = 1; }
    struct timeval now;
    gettimeofday(&now, NULL);
    return now.tv_sec - start.tv_sec;
}

void print_time_pref() {
    printf("%ld ", program_start_sec());
}

// ----- Customer structure -----
typedef struct Customer {
    int id;
    // semaphores/conds to sync with chef
    sem_t sem_served;    // posted by chef when starting to serve (so customer can 'getcake')
    sem_t sem_bake_done;  // posted by chef when baking is finished
    sem_t sem_payment_ok; // posted by chef when payment accepted (allowed to leave)
    struct Customer *next; // for queue linking
} Customer;

// ----- FIFO queue helpers -----
typedef struct Queue {
    Customer *head;
    Customer *tail;
    int size;
} Queue;

void q_init(Queue *q) { q->head = q->tail = NULL; q->size = 0; }
void q_push(Queue *q, Customer *c) {
    c->next = NULL;
    if (!q->tail) q->head = q->tail = c;
    else { q->tail->next = c; q->tail = c; }
    q->size++;
}

Customer* q_pop(Queue *q) {
    if (!q->head) return NULL;
    Customer *c = q->head;
    q->head = c->next;
    if (!q->head) q->tail = NULL;
    c->next = NULL;
}

```

```

    q->size--;
    return c;
}

// ----- Global bakery state -----
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t chef_cv = PTHREAD_COND_INITIALIZER; // chefs wait on this if nothing to do
pthread_cond_t standing_cv = PTHREAD_COND_INITIALIZER; // standing customers wait for
seat
int current_customers = 0; // overall count inside bakery (max MAX_CAPACITY)
int sofa_occupied = 0; // current occupied sofa seats (reserved until customer leaves)

Queue sofa_queue; // customers currently on sofa (FIFO) -> waiting to be served by chefs
Queue standing_queue; // customers standing waiting for an available sofa
Queue payment_queue; // customers who have finished pay action and waiting for chef to
accept payment

// chefs threads will run forever (or until main exits)
int shutdown_flag = 0; // not used for gentle shutdown in this sample

// ----- Logging helpers -----
void log_customer_action(int id, const char *action) {
    print_time_pref();
    printf("Customer %d %s\n", id, action);
    fflush(stdout);
}
void log_chef_action(int id, const char *action) {
    print_time_pref();
    printf("Chef %d %s\n", id, action);
    fflush(stdout);
}

// ----- Chef thread -----
void *chef_thread(void *arg) {
    int chef_id = (int)(intptr_t)arg;

    while (!shutdown_flag) {
        Customer *toPay = NULL;
        Customer *toBake = NULL;

        // pick work: payments have priority
        pthread_mutex_lock(&lock);

```

```

while (payment_queue.size == 0 && sofa_queue.size == 0 && !shutdown_flag) {
    // nothing to do, wait
    pthread_cond_wait(&chef_cv, &lock);
}
if (shutdown_flag) {
    pthread_mutex_unlock(&lock);
    break;
}

// payment priority
if (payment_queue.size > 0) {
    toPay = q_pop(&payment_queue);
    // mark that this chef will accept payment for 'toPay'
    pthread_mutex_unlock(&lock);

    // perform accept payment (2 seconds) - atomic action
    log_chef_action(chef_id, "accepts payment");
    sleep(2);

    // signal customer that payment accepted -> they can leave
    sem_post(&toPay->sem_payment_ok);

    // after finishing payment, continue loop
    continue;
}

// if no payments, check sofa queue to bake
if (sofa_queue.size > 0) {
    toBake = q_pop(&sofa_queue);
    // Note: sofa seat remains reserved for this customer until they leave.
    pthread_mutex_unlock(&lock);

    // signal customer to start getcake concurrently
    sem_post(&toBake->sem_served);

    // Chef bakes (2 seconds)
    log_chef_action(chef_id, "bakes");
    sleep(2);

    // baking finished, inform customer that cake is ready
    sem_post(&toBake->sem_bake_done);
}

```

```

        continue;
    }
    // unlock if fell through
    pthread_mutex_unlock(&lock);
}
return NULL;
}

// ----- Customer thread -----
void *customer_thread(void *arg) {
    Customer *c = (Customer *)arg;

    // enterofficebakery (we simulate action time)
    log_customer_action(c->id, "enters");
    sleep(1); // customer action = 1s

    pthread_mutex_lock(&lock);
    // capacity check done before creating thread — but double-check here:
    if (current_customers >= MAX_CAPACITY) {
        // bakery full, leaves immediately
        pthread_mutex_unlock(&lock);
        log_customer_action(c->id, "leaves");
        // cleanup
        sem_destroy(&c->sem_served);
        sem_destroy(&c->sem_bake_done);
        sem_destroy(&c->sem_payment_ok);
        free(c);
        return NULL;
    }
    // customer is now inside
    current_customers++;

    // try to sit on sofa immediately if seat free
    if (sofa_occupied < SOFA_SEATS) {
        // take seat
        sofa_occupied++;
        pthread_mutex_unlock(&lock);

        // sitOnSofa action (1 sec)
        log_customer_action(c->id, "sits");
        sleep(1);
    }
}

```



```

    // join sofa queue (wait to be served)
    pthread_mutex_lock(&lock);
    q_push(&sofa_queue, c);
    pthread_cond_broadcast(&chef_cv); // signal chefs that work available
    pthread_mutex_unlock(&lock);
} else {
    // sofa full -> stand
    // join standing_queue and wait for a seat
    q_push(&standing_queue, c);
    log_customer_action(c->id, "stands");
    // wait until signaled that a seat is free and it's our turn (FIFO)
    while (1) {
        pthread_cond_wait(&standing_cv, &lock);
        // when signaled, check if we are at head of standing queue and if a sofa seat is available
        if (standing_queue.head == c && sofa_occupied < SOFA_SEATS) {
            // remove from standing_queue head (we are head)
            Customer *popped = q_pop(&standing_queue);
            if (popped != c) {
                // shouldn't happen
                fprintf(stderr, "Error: standing queue mismatch\n");
            }
            // occupy seat
            sofa_occupied++;
            // release lock and do sit action
            pthread_mutex_unlock(&lock);

            // sitOnSofa action
            log_customer_action(c->id, "sits");
            sleep(1);

            // join sofa queue
            pthread_mutex_lock(&lock);
            q_push(&sofa_queue, c);
            pthread_cond_broadcast(&chef_cv);
            pthread_mutex_unlock(&lock);
            break;
        }
        // otherwise continue waiting
    }
}

// now wait to be served: chef will sem_post(sem_served) to indicate they started serving (so that

```

getcake runs concurrently)

```
sem_wait(&c->sem_served);
```

// getcake (1 sec) - must happen concurrently with chef's bake

```
log_customer_action(c->id, "requests cake");
```

```
sleep(1);
```

// wait for chef to complete baking (chef posts sem_bake_done after 2s)

```
sem_wait(&c->sem_bake_done);
```

// after cake finished -> pay (1s)

```
log_customer_action(c->id, "pays");
```

```
sleep(1);
```

// after pay action is done, we add ourselves to payment_queue and wait for an available chef to accept payment

```
pthread_mutex_lock(&lock);
```

```
q_push(&payment_queue, c);
```

```
pthread_cond_broadcast(&chef_cv); // notify chefs (they prioritize payment)
```

```
pthread_mutex_unlock(&lock);
```

// wait until some chef accepts and posts sem_payment_ok

```
sem_wait(&c->sem_payment_ok);
```

// now payment has been accepted by a chef (chef spent 2s)

// customer leaves now; free sofa seat and decrement capacity

```
pthread_mutex_lock(&lock);
```

```
sofa_occupied--;
```

```
current_customers--;
```

// free a sofa seat, so if anybody is standing, wake them (the earliest standing customer)

```
pthread_cond_broadcast(&standing_cv);
```

// also notify chefs in case they were waiting for tasks

```
pthread_cond_broadcast(&chef_cv);
```

```
pthread_mutex_unlock(&lock);
```

```
log_customer_action(c->id, "leaves");
```

// cleanup

```
sem_destroy(&c->sem_served);
```

```
sem_destroy(&c->sem_bake_done);
```

```
sem_destroy(&c->sem_payment_ok);
```

```
free(c);
```

```
return NULL;
```

```

}

// ----- Input scheduler -----
typedef struct Arrival {
    int ts;
    int id;
    struct Arrival *next;
} Arrival;

int main() {
    // initialize queues
    q_init(&sofa_queue);
    q_init(&standing_queue);
    q_init(&payment_queue);

    // start chef threads
    pthread_t chefs[NUM_CHEFS];
    for (int i = 0; i < NUM_CHEFS; ++i) {
        pthread_create(&chefs[i], NULL, chef_thread, (void*)(intptr_t)(i+1));
    }

    // Read arrival lines until <EOF>
    // Format: "<timestamp> Customer <id>"
    char line[256];
    Arrival *arr_head = NULL, *arr_tail = NULL;
    while (fgets(line, sizeof(line), stdin)) {
        if (strncmp(line, "<EOF>", 5) == 0) break;
        if (strlen(line) == 0) continue;
        int ts, id;
        char word[64];
        if (sscanf(line, "%d %63s %d", &ts, word, &id) == 3) {
            if (strcmp(word, "Customer") != 0) {
                fprintf(stderr, "Skipping invalid line: %s", line);
                continue;
            }
            Arrival *a = malloc(sizeof(Arrival));
            a->ts = ts;
            a->id = id;
            a->next = NULL;
            if (!arr_tail) arr_head = arr_tail = a;
            else { arr_tail->next = a; arr_tail = a; }
        } else {

```

```

    fprintf(stderr, "Skipping invalid line: %s", line);
}
}

// schedule customers according to timestamps
// we'll use a simple sleep-until-next-arrival strategy
// program start time is captured implicitly by first call to program_start_sec()
gettimeofday(&(struct timeval){0}, NULL); // just to ensure start time established by function

Arrival *cur = arr_head;
long start_time = program_start_sec(); (void)start_time;

while (cur) {
    int target = cur->ts;
    // sleep until target second (relative)
    long now = program_start_sec();
    if (target > now) sleep(target - now);

    // create customer thread if capacity not exceeded at creation moment
    pthread_mutex_lock(&lock);
    if (current_customers >= MAX_CAPACITY) {
        // customer finds bakery full and leaves immediately (no thread created)
        pthread_mutex_unlock(&lock);
        print_time_pref();
        printf("Customer %d leaves\n", cur->id);
        fflush(stdout);
        Arrival *tmp = cur;
        cur = cur->next;
        free(tmp);
        continue;
    }
    // else create Customer struct and thread
    Customer *c = malloc(sizeof(Customer));
    c->id = cur->id;
    sem_init(&c->sem_served, 0, 0);
    sem_init(&c->sem_bake_done, 0, 0);
    sem_init(&c->sem_payment_ok, 0, 0);
    c->next = NULL;
    pthread_t tid;
    if (pthread_create(&tid, NULL, customer_thread, c) != 0) {
        perror("pthread_create customer");
        exit(1);
    }
}

```

```

    }

    // detach customer thread
    pthread_detach(tid);
    pthread_mutex_unlock(&lock);

    Arrival *tmp = cur;
    cur = cur->next;
    free(tmp);
}

// after scheduling all arrivals, wait a little for processing to finish
// (in a real assignment you might join threads or have a shutdown protocol)
// we'll sleep long enough for all work to be done (rough heuristic)
// But avoid indefinite sleep: wait up to a generous timeout.
sleep(30);

// request shutdown (not strictly necessary)
pthread_mutex_lock(&lock);
shutdown_flag = 1;
pthread_cond_broadcast(&chef_cv);
pthread_mutex_unlock(&lock);

for (int i = 0; i < NUM_CHEFS; ++i) pthread_join(chefs[i], NULL);

return 0;
}

```

Notes / Design decisions & assumptions

- **Timestamps:** Program treats arrival timestamps as seconds relative to program start and spawns customer threads at those seconds. When printing each action we print the elapsed seconds since the program started (integer seconds).
- **Sofa reservation semantics:** Once a customer sits on the sofa, that seat is reserved until the customer leaves (after payment accepted). While being served (baking / payment), they keep their seat — this matches the problem statement.
- **Payment ordering & priority:** After cake finishes, customer performs `pays` (1s) and then enqueues themselves into `payment_queue`. Chefs pick from `payment_queue` first (priority), accepting payments (2s). A customer's leave is only allowed after `acceptPayment` for them finishes.

- **Chef/cust synchronization:** When a chef picks a sofa-customer to bake, they `sem_post(sem_served)` so the customer starts `getcake` concurrently; chef sleeps 2s while baking and posts `sem_bake_done` when finished.
 - **Action durations:** Customer actions = 1s each (enter, sit, getcake, pay). Chef actions (bake, accept payment) = 2s each. These sleeps are not interruptible.
 - **Simple shutdown:** Program sleeps for 30s after scheduling all arrivals to allow processing to finish; adjust if you have larger test inputs. This can be replaced by a more elaborate join/shutdown if desired.
-

Test inputs (notorious / stress cases)

Below are several test input scenarios you can save and run. Each demonstrates tricky behaviour (capacity boundary, many simultaneous arrivals, standing -> sofa handover, payment-priority starvation risk avoided by design):

Test 1 — Simple flow (basic)

yaml

```
1 Customer 1001
2 Customer 1002
3 Customer 1003
4 Customer 1004
5 Customer 1005
<EOF>
```

Expected: first 4 sit on sofa; 5th stands until a seat frees after someone leaves.

Test 2 — Stressing capacity (many arrivals rapidly)

php-template

```
0 Customer 1
0 Customer 2
0 Customer 3
0 Customer 4
0 Customer 5
0 Customer 6
0 Customer 7
0 Customer 8
```

```
0 Customer 9
0 Customer 10
0 Customer 11
0 Customer 12
0 Customer 13
0 Customer 14
0 Customer 15
0 Customer 16
0 Customer 17
0 Customer 18
0 Customer 19
0 Customer 20
0 Customer 21
0 Customer 22
0 Customer 23
0 Customer 24
0 Customer 25
0 Customer 26
<EOF>
```

Expected: first 25 enter; 26th leaves immediately (capacity). Sofa seats only 4 — others stand.

Test 3 — Payment priority interplay (arrivals timed to create pending payments)

```
yaml

1 Customer 2001
1 Customer 2002
2 Customer 2003
2 Customer 2004
3 Customer 2005
4 Customer 2006
5 Customer 2007
6 Customer 2008
<EOF>
```

This times multiple arrivals so chefs will alternate baking and accepting payment. Useful to observe chefs picking payments before new bakes.

Test 4 — Simultaneous arrival at capacity boundary (to show leaves)

yaml

```
1 Customer 3001
1 Customer 3002
1 Customer 3003
1 Customer 3004
1 Customer 3005
1 Customer 3006
1 Customer 3007
1 Customer 3008
1 Customer 3009
1 Customer 3010
1 Customer 3011
1 Customer 3012
1 Customer 3013
1 Customer 3014
1 Customer 3015
1 Customer 3016
1 Customer 3017
1 Customer 3018
1 Customer 3019
1 Customer 3020
1 Customer 3021
1 Customer 3022
1 Customer 3023
1 Customer 3024
1 Customer 3025
1 Customer 3026
<EOF>
```

Many arriving exactly at time `1`: the last ones beyond 25 should leave.

If you'd like, I can:

- Modify the program to print **when** chefs finish baking and when they finish accepting payment (currently it prints action start).
- Replace the simple `sleep(30)` finish-wait with a robust join/shutdown so the program exits only after all customers complete.

- Adapt the timestamps to milliseconds or a simulated logical clock (instead of `sleep`) for fast unit testing.
- Produce a single combined test file that exercises all tricky cases and run it, capturing example output.

Which of those would you like next?

In file included from queue.c:1:

shop.h:25:8: error: unknown type name 'Queue'

```
25 | extern Queue sofa_queue; // customers
    |          ^~~~~
currently on sofa (FIFO) -> waiting to be served by
chefs
```

```
|          ^~~~~
```

shop.h:26:8: error: unknown type name 'Queue'

```
26 | extern Queue standing_queue; // customers
    |          ^~~~~
standing waiting for an available sofa
```

```
|          ^~~~~
```

shop.h:27:8: error: unknown type name 'Queue'

```
27 | extern Queue payment_queue; // customers
    |          ^~~~~
who have finished pay action and waiting for chef to
accept payment
```

```
|          ^~~~~
```

In file included from threads.c:2:

shop.h:25:8: error: unknown type name 'Queue'

```
25 | extern Queue sofa_queue; // customers
    |          ^~~~~
currently on sofa (FIFO) -> waiting to be served by
chefs
```

```
|          ^~~~~
```

shop.h:26:8: error: unknown type name 'Queue'

```
26 | extern Queue standing_queue; // customers
    |          ^~~~~
standing waiting for an available sofa
```

```
|          ^~~~~
```

shop.h:27:8: error: unknown type name 'Queue'

```
27 | extern Queue payment_queue; // customers
    |          ^~~~~
who have finished pay action and waiting for chef to
accept payment
```

```
|          ^~~~~
```

threads.c: In function 'chef_thread':

threads.c:14:29: error: request for member 'size' in something not a structure or union

```
14 |         while (payment_queue.size == 0 &&
```

```

sofa_queue.size == 0 && !shutdown_flag) {
    |
    ^
threads.c:14:53: error: request for member 'size' in
something not a structure or union
14 |     while (payment_queue.size == 0 &&
sofa_queue.size == 0 && !shutdown_flag) {
    |
    ^
threads.c:24:26: error: request for member 'size' in
something not a structure or union
24 |     if (payment_queue.size > 0) {
    |
    ^
threads.c:25:27: warning: passing argument 1 of
'q_pop' from incompatible pointer type [-
Wincompatible-pointer-types]
25 |     toPay = q_pop(&payment_queue);
    |
    ^~~~~~
    |
    int *
shop.h:61:24: note: expected 'Queue *' but argument is
of type 'int *'
61 | Customer* q_pop(Queue *q);
    |
    ~~~~~~^
threads.c:41:23: error: request for member 'size' in
something not a structure or union
41 |     if (sofa_queue.size > 0) {
    |
    ^
threads.c:42:28: warning: passing argument 1 of
'q_pop' from incompatible pointer type [-
Wincompatible-pointer-types]
42 |     toBake = q_pop(&sofa_queue);
    |
    ^~~~~~
    |
    int *
shop.h:61:24: note: expected 'Queue *' but argument is
of type 'int *'
61 | Customer* q_pop(Queue *q);
    |
    ~~~~~~^
threads.c: In function 'customer_thread':
threads.c:100:16: warning: passing argument 1 of
'q_push' from incompatible pointer type [-
Wincompatible-pointer-types]
100 |     q_push(&sofa_queue, c);

```

```

|           ^~~~~~
|           |
|           int *

```

shop.h:60:20: note: expected 'Queue *' but argument is of type 'int *'

```

60 | void q_push(Queue *q, Customer *c);
|           ~~~~~^

```

threads.c:106:16: warning: passing argument 1 of 'q_push' from incompatible pointer type [-Wincompatible-pointer-types]

```

106 |     q_push(&standing_queue, c);
|           ^~~~~~
|           |
|           int *

```

shop.h:60:20: note: expected 'Queue *' but argument is of type 'int *'

```

60 | void q_push(Queue *q, Customer *c);
|           ~~~~~^

```

threads.c:112:31: error: request for member 'head' in something not a structure or union

```

112 |     if (standing_queue.head == c &&
    |                  ^
    sofa_occupied < SOFA_SEATS) {

```

threads.c:114:42: warning: passing argument 1 of 'q_pop' from incompatible pointer type [-Wincompatible-pointer-types]

```

114 |     Customer *popped =
    |     q_pop(&standing_queue);
    |           ^~~~~~
    |           |
    |           int *

```

shop.h:61:24: note: expected 'Queue *' but argument is of type 'int *'

```

61 | Customer* q_pop(Queue *q);
|           ~~~~~^

```

threads.c:130:24: warning: passing argument 1 of 'q_push' from incompatible pointer type [-Wincompatible-pointer-types]

```

130 |     q_push(&sofa_queue, c);
|           ^~~~~~
|           |
|           int *

```

shop.h:60:20: note: expected 'Queue *' but argument is of type 'int *'

```
60 | void q_push(Queue *q, Customer *c);  
    |           ~~~~~^
```

threads.c:154:12: warning: passing argument 1 of 'q_push' from incompatible pointer type [-Wincompatible-pointer-types]

```
154 |   q_push(&payment_queue, c);  
      |       ^~~~~~  
      |       |  
      |     int *
```

shop.h:60:20: note: expected 'Queue *' but argument is of type 'int *'

```
60 | void q_push(Queue *q, Customer *c);  
    |           ~~~~~^
```

In file included from utils.c:1:

shop.h:25:8: error: unknown type name 'Queue'

```
25 | extern Queue sofa_queue; // customers  
    |                ^~~~~  
currently on sofa (FIFO) -> waiting to be served by  
chefs
```

shop.h:26:8: error: unknown type name 'Queue'

```
26 | extern Queue standing_queue; // customers  
    |                ^~~~~  
standing waiting for an available sofa
```

shop.h:27:8: error: unknown type name 'Queue'

```
27 | extern Queue payment_queue; // customers  
    |                ^~~~~  
who have finished pay action and waiting for chef to  
accept payment
```

In file included from main.c:1:

shop.h:25:8: error: unknown type name 'Queue'

```
25 | extern Queue sofa_queue; // customers  
    |                ^~~~~  
currently on sofa (FIFO) -> waiting to be served by  
chefs
```

shop.h:26:8: error: unknown type name 'Queue'

```
26 | extern Queue standing_queue; // customers  
    |                ^~~~~  
standing waiting for an available sofa
```

shop.h:27:8: error: unknown type name 'Queue'

```
27 | extern Queue payment_queue; // customers
```

```

who have finished pay action and waiting for chef to
accept payment
    |      ^~~~~
main.c:10:7: error: conflicting types for 'sofa_queue';
have 'Queue'
    10 | Queue sofa_queue;    // customers currently on
sofa (FIFO) -> waiting to be served by chefs
    |      ^~~~~~
shop.h:25:14: note: previous declaration of 'sofa_queue'
with type 'int'
    25 | extern Queue sofa_queue;    // customers
currently on sofa (FIFO) -> waiting to be served by
chefs
    |      ^~~~~~
main.c:11:7: error: conflicting types for
'standing_queue'; have 'Queue'
    11 | Queue standing_queue; // customers standing
waiting for an available sofa
    |      ^~~~~~
shop.h:26:14: note: previous declaration of
'standing_queue' with type 'int'
    26 | extern Queue standing_queue; // customers
standing waiting for an available sofa
    |      ^~~~~~
main.c:12:7: error: conflicting types for
'payment_queue'; have 'Queue'
    12 | Queue payment_queue; // customers who have
finished pay action and waiting for chef to accept
payment
    |      ^~~~~~
shop.h:27:14: note: previous declaration of
'payment_queue' with type 'int'
    27 | extern Queue payment_queue; // customers
who have finished pay action and waiting for chef to
accept payment
    |      ^~~~~~
main.c:15:5: error: redefinition of 'shutdown_flag'
    15 | int shutdown_flag = 0; // not used for gentle
shutdown in this sample
    |      ^~~~~~
shop.h:30:5: note: previous definition of
'shutdown_flag' with type 'int'

```

```
30 | int shutdown_flag = 0; // not used for gentle
    | shutdown in this sample
```

```
    | ^~~~~~
make: *** [Makefile:2: all] Error 1
```

how do i fix this?

Got it 👍 — this is a **type-definition and header organization issue**.

Right now your compiler is yelling because:

1. `Queue` is **not defined** before you use it in `shop.h`.
2. In `shop.h`, you accidentally declared the global queues (`extern Queue sofa_queue`) *but without having the `Queue` type visible*. The compiler then assumes `Queue` is an `int` → hence the later “conflicting types” error.
3. You also **defined** `shutdown_flag` **twice** — once in `shop.h` and once in `main.c`.

✅ How to fix step by step

1. Define `Queue` struct in `shop.h`

Make sure `Queue` is declared in `shop.h` **before** you use it.

```
c

// shop.h
#ifndef SHOP_H
#define SHOP_H

#include <pthread.h>

#define SOFA_SEATS 4
#define MAX_CAPACITY 25

typedef struct Customer {
    int id;
    int arrival_time;
    struct Customer *next;
} Customer;

typedef struct Queue {
    Customer *head;
```

```

    Customer *tail;
    int size;
    pthread_mutex_t lock;
} Queue;

// Declare queues as extern (definition will be in main.c)
extern Queue sofa_queue;
extern Queue standing_queue;
extern Queue payment_queue;

extern int shutdown_flag;

// Queue functions
void q_init(Queue *q);
void q_push(Queue *q, Customer *c);
Customer* q_pop(Queue *q);

#endif

```

2. Define queues once in `main.c`

```

c

#include "shop.h"

Queue sofa_queue;
Queue standing_queue;
Queue payment_queue;

int shutdown_flag = 0; // Define here ONLY

```

3. Remove `int shutdown_flag = 0;` from `shop.h`

In `shop.h`, only declare:

```

c

extern int shutdown_flag;

```

4. Queue functions in `queue.c`

```
c

#include "shop.h"
#include <stdlib.h>

void q_init(Queue *q) {
    q->head = q->tail = NULL;
    q->size = 0;
    pthread_mutex_init(&q->lock, NULL);
}

void q_push(Queue *q, Customer *c) {
    c->next = NULL;
    pthread_mutex_lock(&q->lock);
    if (q->tail) {
        q->tail->next = c;
        q->tail = c;
    } else {
        q->head = q->tail = c;
    }
    q->size++;
    pthread_mutex_unlock(&q->lock);
}

Customer* q_pop(Queue *q) {
    pthread_mutex_lock(&q->lock);
    Customer *c = q->head;
    if (c) {
        q->head = c->next;
        if (!q->head) q->tail = NULL;
        q->size--;
    }
    pthread_mutex_unlock(&q->lock);
    return c;
}
```



Key fixes you needed:

- Define `queue` before using it.

- Use `extern` in header, and define **once** in `main.c`.
 - Avoid global variable **redefinition** (`shutdown_flag`).
-

👉 Do you want me to also **give you a corrected Makefile** + minimal runnable `main.c` (with test input loop) so you can compile and run end-to-end?

```
0 Customer 1 enters
0 Customer 2 enters
0 Customer 4 enters
0 Customer 3 enters
0 Customer 5 enters
0 Customer 6 enters
0 Customer 7 enters
0 Customer 8 enters
0 Customer 9 enters
0 Customer 10 enters
0 Customer 11 enters
0 Customer 12 enters
0 Customer 13 enters
0 Customer 14 enters
0 Customer 15 enters
0 Customer 16 enters
0 Customer 17 enters
0 Customer 18 enters
0 Customer 19 enters
0 Customer 20 enters
0 Customer 21 enters
0 Customer 22 enters
0 Customer 23 enters
0 Customer 24 enters
0 Customer 25 enters
0 Customer 26 enters
1 Customer 2 sits
1 Customer 1 sits
1 Customer 4 sits
1 Customer 3 sits
1 Customer 5 stands
1 Customer 13 stands
1 Customer 10 stands
```

1 Customer 8 stands
1 Customer 22 stands
1 Customer 23 stands
1 Customer 11 stands
1 Customer 16 stands
1 Customer 18 stands
1 Customer 15 stands
1 Customer 19 stands
1 Customer 6 stands
1 Customer 20 stands
1 Customer 17 stands
1 Customer 9 stands
1 Customer 7 stands
1 Customer 25 stands
1 Customer 26 stands
1 Customer 12 stands
1 Customer 21 stands
1 Customer 14 stands
1 Customer 24 leaves
2 Chef 3 bakes
2 Customer 2 requests cake
2 2 Chef 2 bakes
2 Customer 1 requests cake
Customer 3 requests cake
2 Customer 4 requests cake
2 Chef 4 bakes
2 Chef 1 bakes
4 Customer 2 pays
4 Customer 1 pays
4 Customer 3 pays
4 Customer 4 pays
5 Chef 3 accepts payment
5 Chef 4 accepts payment
5 Chef 1 accepts payment
5 Chef 2 accepts payment
7 Customer 2 leaves
7 Customer 3 leaves
7 Customer 1 leaves
7 Customer 5 sits
7 Customer 4 leaves
7 Customer 10 sits
7 Customer 13 sits

7 Customer 8 sits
8 Customer 5 requests cake
Chef 4 bakes
8 Chef 3 bakes
8 Customer 8 requests cake
8 Chef 2 bakes
8 Customer 10 requests cake
8 Chef 1 bakes
8 Customer 13 requests cake
10 Customer 5 pays
10 Customer 8 pays
10 Customer 10 pays
10 Customer 13 pays
11 Chef 3 accepts payment
11 Chef 4 accepts payment
11 Chef 2 accepts payment
11 Chef 1 accepts payment
13 Customer 5 leaves
13 Customer 22 sits
13 Customer 8 leaves
13 Customer 13 leaves
13 Customer 10 leaves
13 Customer 23 sits
14 Chef 1 bakes
14 Customer 22 requests cake
14 Chef 3 bakes
14 Customer 23 requests cake
16 Customer 22 pays
16 Customer 23 pays
17 Chef 1 accepts payment
17 Chef 3 accepts payment
19 Customer 22 leaves
19 Customer 11 sits
19 Customer 16 sits
19 Customer 23 leaves
19 Customer 18 sits
20 Chef 3 bakes
20 Customer 11 requests cake
20 Chef 2 bakes
20 Customer 16 requests cake
20 Chef 1 bakes
20 Customer 18 requests cake

22 Customer 11 pays
22 Customer 16 pays
22 Customer 18 pays
23 Chef 2 accepts payment
23 Chef 3 accepts payment
23 Chef 4 accepts payment
25 Customer 16 leaves
25 Customer 11 leaves
25 Customer 15 sits
25 Customer 19 sits
25 Customer 18 leaves
25 Customer 6 sits
25 Customer 20 sits
26 26 Customer 19 requests cake
Chef 3 bakes
26 Chef 2 bakes
26 Customer 15 requests cake
26 Chef 4 bakes
26 Chef 1 bakes
26 26 Customer 6 requests cake
Customer 20 requests cake
28 Customer 15 pays
28 Customer 19 pays
28 Customer 20 pays
28 Customer 6 pays
29 Chef 3 accepts payment
29 Chef 2 accepts payment
29 Chef 4 accepts payment
29 Chef 1 accepts payment
31 Customer 19 leaves
31 Customer 17 sits
31 Customer 15 leaves
31 Customer 9 sits
31 Customer 7 sits
31 Customer 6 leaves
31 Customer 20 leaves

why does it end when sm customers are just sitting?

Good catch 👍 — the reason your output **stops even though some customers are still sitting** is because of how your **termination condition** (`shutdown_flag`) is handled.

From the trace, you can see:

- At the end, Customers 17, 9, and 7 are *sitting* but they never:
 - request cake
 - pay
 - leave

Then the simulation ends. That usually means:

🔍 What's happening

1. **Your main thread sets `shutdown_flag` too early.**
 - Likely after all customers have *arrived* (entered the shop), you signal shutdown.
 - But sitting customers are still waiting for service → so threads exit before finishing them.
2. **Chefs and customers might be waiting on queues that are already considered “closed.”**
 - If your loop is like:

```
c

while (!shutdown_flag) {
    // process queues
}
```

then once `shutdown_flag` flips, the threads stop even if `sofa_queue` or `payment_queue` is non-empty.

3. **No “drain the queues” step.**
 - Proper simulation should:
 - Stop arrivals once time ends.
 - But let all queues empty out naturally before shutting down.
-

✅ How to fix it

Instead of killing threads when arrivals end, do this:

1. Change shutdown condition

Chefs/customers should run while **there is still work left**:

```
c

while (!(shutdown_flag && q->size == 0)) {
    // keep baking / serving while queue has people
}
```

So the check becomes:

- If shutdown has started AND queues are empty → exit.
 - Otherwise → keep working.
-

2. Main thread termination

- Don't set `shutdown_flag` right after the last customer arrives.
- Wait until **all customers leave**:
 - Track number of customers that entered vs. number that left.
 - Example:

```
c

int total_entered = N;
int total_left = 0;

// Each customer thread increments total_left when leaving
if (total_left == total_entered) shutdown_flag = 1;
```

3. Optional cleanup

If you want a strict end:

- `pthread_join` all customer and chef threads.
 - Then exit.
-

 **In short**

Right now your simulation ends when **arrivals are done**, not when **service is finished**.

You need to **delay shutdown until all customers finish paying + leaving**.

👉 Do you want me to rewrite your **main.c control loop** to include:

- `total_customers_entered` / `total_customers_left` counters
- proper `while` conditions for chefs/customers
so the program only ends when everyone leaves?

```
#include "shop.h"

// ----- Chef thread -----
void *chef_thread(void *arg) {
    int chef_id = (int)(intptr_t)arg;

    while (!shutdown_flag) {
        Customer *toPay = NULL;
        Customer *toBake = NULL;

        // pick work: payments have priority
        pthread_mutex_lock(&lock);
        while (payment_queue.size == 0 &&
            sofa_queue.size == 0 && !shutdown_flag) {
            // nothing to do, wait
            pthread_cond_wait(&chef_cv, &lock);
        }
        if (shutdown_flag) {
            pthread_mutex_unlock(&lock);
            break;
        }

        // payment priority
        if (payment_queue.size > 0) {
            toPay = q_pop(&payment_queue);
            // mark that this chef will accept payment for
            'toPay'
            pthread_mutex_unlock(&lock);
```

```

        // perform accept payment (2 seconds) - atomic
action
        log_chef_action(chef_id, "accepts payment");
        sleep(2);

        // signal customer that payment accepted ->
they can leave
        sem_post(&toPay->sem_payment_ok);

        // after finishing payment, continue loop
        continue;
    }

    // if no payments, check sofa queue to bake
    if (sofa_queue.size > 0) {
        toBake = q_pop(&sofa_queue);
        // Note: sofa seat remains reserved for this
customer until they leave.
        pthread_mutex_unlock(&lock);

        // signal customer to start getcake concurrently
        sem_post(&toBake->sem_served);

        // Chef bakes (2 seconds)
        log_chef_action(chef_id, "bakes");
        sleep(2);

        // baking finished, inform customer that cake is
ready
        sem_post(&toBake->sem_bake_done);

        continue;
    }
    // unlock if fell through
    pthread_mutex_unlock(&lock);
}
return NULL;
}

// ----- Customer thread -----
void *customer_thread(void *arg) {

```



```

Customer *c = (Customer *)arg;

// enterofficebakery (we simulate action time)
log_customer_action(c->id, "enters");
sleep(1); // customer action = 1s

pthread_mutex_lock(&lock);
// capacity check done before creating thread — but
double-check here:
if (current_customers >= MAX_CAPACITY) {
    // bakery full, leaves immediately
    pthread_mutex_unlock(&lock);
    log_customer_action(c->id, "leaves");
    // cleanup
    sem_destroy(&c->sem_served);
    sem_destroy(&c->sem_bake_done);
    sem_destroy(&c->sem_payment_ok);
    free(c);
    return NULL;
}
// customer is now inside
current_customers++;

// try to sit on sofa immediately if seat free
if (sofa_occupied < SOFA_SEATS) {
    // take seat
    sofa_occupied++;
    pthread_mutex_unlock(&lock);

    // sitOnSofa action (1 sec)
    log_customer_action(c->id, "sits");
    sleep(1);

    // join sofa queue (wait to be served)
    pthread_mutex_lock(&lock);
    q_push(&sofa_queue, c);
    pthread_cond_broadcast(&chef_cv); // signal chefs
that work available
    pthread_mutex_unlock(&lock);
} else {
    // sofa full -> stand
    // join standing_queue and wait for a seat

```

```

    q_push(&standing_queue, c);
    log_customer_action(c->id, "stands");
    // wait until signaled that a seat is free and it's our
    turn (FIFO)
    while (1) {
        pthread_cond_wait(&standing_cv, &lock);
        // when signaled, check if we are at head of
        standing queue and if a sofa seat is available
        if (standing_queue.head == c && sofa_occupied
        < SOFA_SEATS) {
            // remove from standing_queue head (we are
            head)
            Customer *popped =
            q_pop(&standing_queue);
            if (popped != c) {
                // shouldn't happen
                fprintf(stderr, "Error: standing queue
                mismatch\n");
            }
            // occupy seat
            sofa_occupied++;
            // release lock and do sit action
            pthread_mutex_unlock(&lock);

            // sitOnSofa action
            log_customer_action(c->id, "sits");
            sleep(1);

            // join sofa queue
            pthread_mutex_lock(&lock);
            q_push(&sofa_queue, c);
            pthread_cond_broadcast(&chef_cv);
            pthread_mutex_unlock(&lock);
            break;
        }
        // otherwise continue waiting
    }
}

// now wait to be served: chef will
sem_post(sem_served) to indicate they started serving
(so that getcake runs concurrently)

```

```

sem_wait(&c->sem_served);
// getcake (1 sec) - must happen concurrently with
chef's bake
log_customer_action(c->id, "requests cake");
sleep(1);

// wait for chef to complete baking (chef posts
sem_bake_done after 2s)
sem_wait(&c->sem_bake_done);

// after cake finished -> pay (1s)
log_customer_action(c->id, "pays");
sleep(1);

// after pay action is done, we add ourselves to
payment_queue and wait for an available chef to
accept payment
pthread_mutex_lock(&lock);
q_push(&payment_queue, c);
pthread_cond_broadcast(&chef_cv); // notify chefs
(they prioritize payment)
pthread_mutex_unlock(&lock);

// wait until some chef accepts and posts
sem_payment_ok
sem_wait(&c->sem_payment_ok);

// now payment has been accepted by a chef (chef
spent 2s)
// customer leaves now; free sofa seat and
decrement capacity
pthread_mutex_lock(&lock);
sofa_occupied--;
current_customers--;
// free a sofa seat, so if anybody is standing, wake
them (the earliest standing customer)
pthread_cond_broadcast(&standing_cv);
// also notify chefs in case they were waiting for
tasks
pthread_cond_broadcast(&chef_cv);
pthread_mutex_unlock(&lock);

```

```

log_customer_action(c->id, "leaves");

// cleanup
sem_destroy(&c->sem_served);
sem_destroy(&c->sem_bake_done);
sem_destroy(&c->sem_payment_ok);
free(c);
return NULL;
}

```

what seems to be the problem?

You're very close — the root cause is your **shutdown logic + wake-up check** (and a couple of related concurrency pitfalls). Concretely:

Primary bug (why it ends with customers still sitting)

In `chef_thread` you do:

```

c

pthread_mutex_lock(&lock);
while (payment_queue.size == 0 && sofa_queue.size == 0 && !shutdown_flag) {
    pthread_cond_wait(&chef_cv, &lock);
}
if (shutdown_flag) {
    pthread_mutex_unlock(&lock);
    break;
}

```

That `if (shutdown_flag)` unconditionally **breaks the chef loop as soon as** `shutdown_flag` **is set**, even if there *is* still work in `payment_queue` or `sofa_queue`. In other words: main sets `shutdown_flag` when arrivals are done, chefs see `shutdown_flag==true` and exit — leaving sitting customers unserved. That matches your trace where customers are still sitting when the program ends.

Secondary / related issues to fix while we're here

1. Decide who owns locking for queue ops.

- If `q_push` / `q_pop` lock the queue internally, *don't* also hold the global `lock` while calling them (double-lock risk or inconsistent lock ordering).

- If callers are expected to hold `lock` / `queue` lock, make that contract clear. Mixing both styles causes subtle races or deadlocks.
2. **Shutdown condition should wait for all work to finish.**
 - Don't set `shutdown_flag` just because arrivals finished. Instead set it when `total_left == total_arrived` (or when `active_customers == 0`) and all queues are empty. Only then wake chefs to exit.
 3. **Chef loop: check queues after wake and only exit when both shutdown & queues empty.**
 4. **Signalling:** always `pthread_cond_broadcast(&chef_cv)` after enqueueing work.
-

Minimal, safe fixes

A — Fix `chef_thread` loop

Replace your top-of-loop logic with this (assumes `lock` protects queue sizes and `q_pop` is safe to call with `lock` held — adjust if your `q_*` functions do internal locking):

```
c

pthread_mutex_lock(&lock);
while (payment_queue.size == 0 && sofa_queue.size == 0) {
    if (shutdown_flag) {
        // no work and shutdown requested -> exit
        pthread_mutex_unlock(&lock);
        return NULL;
    }
    pthread_cond_wait(&chef_cv, &lock);
}

/* at this point there is work (or we fell out because of shutdown check handled above) */
if (payment_queue.size > 0) {
    toPay = q_pop(&payment_queue);
    pthread_mutex_unlock(&lock);
    // accept payment...
    ...
    continue;
}

if (sofa_queue.size > 0) {
```

```

    toBake = q_pop(&sofa_queue);
    pthread_mutex_unlock(&lock);
    // bake ...

    ...
    continue;
}

pthread_mutex_unlock(&lock);

```

Key change: chef waits only while *both* queues are empty, and exits **only** when `shutdown_flag` AND queues are empty. The previous `if (shutdown_flag) break;` before checking queues was too eager.

B — Better termination condition in `main` (recommended)

Track counts:

- `int total_arrived = 0;` // increment when a customer thread actually enters (or when you spawn)
- `int total_left = 0;` // increment when customer thread logs leaving (inside `customer_thread` while holding `lock`)

After you read and spawn all arrivals, **do not** set `shutdown_flag` immediately.

Instead:

```

c

// in customer_thread, after log_customer_action(c->id, "leaves"):
pthread_mutex_lock(&lock);
total_left++;
if (total_left == total_arrived) {
    shutdown_flag = 1;
    pthread_cond_broadcast(&chef_cv);
}
pthread_mutex_unlock(&lock);

```

This ensures `shutdown_flag` is set only after everyone left. If you prefer to set shutdown when arrivals are done *and* queues empty, you can check `sofa_queue.size==0 && payment_queue.size==0 && standing_queue.size==0` as well.

C — If your queue ops lock internally

If `q_push` / `q_pop` internally do `pthread_mutex_lock(&q->lock)` then:

- Either stop holding `lock` when calling them (use consistent order to avoid deadlocks), OR
- Change `q_*` to *not* lock internally and make the caller responsible for locking.

A simple consistent approach: keep a single global `lock` that protects all shared state (queues, counts). Make `q_push` / `q_pop` **not** use per-queue locks and assume the caller holds `lock`. That reduces complexity for this project.

Example: chef loop (final snippet)

This one assumes caller holds `lock` and `q_pop` / `q_push` do not lock internally:

c

```
void *chef_thread(void *arg) {
    int chef_id = (int)(intptr_t)arg;

    while (1) {
        Customer *toPay = NULL;
        Customer *toBake = NULL;

        pthread_mutex_lock(&lock);
        while (payment_queue.size == 0 && sofa_queue.size == 0) {
            if (shutdown_flag) {
                pthread_mutex_unlock(&lock);
                return NULL; // graceful exit
            }
            pthread_cond_wait(&chef_cv, &lock);
        }

        if (payment_queue.size > 0) {
            toPay = q_pop(&payment_queue); // caller holds lock
            pthread_mutex_unlock(&lock);

            log_chef_action(chef_id, "accepts payment");
            sleep(2);
            sem_post(&toPay->sem_payment_ok);
            continue;
        }

        if (sofa_queue.size > 0) {
```

```

    toBake = q_pop(&sofa_queue);
    pthread_mutex_unlock(&lock);

    sem_post(&toBake->sem_served);
    log_chef_action(chef_id, "bakes");
    sleep(2);
    sem_post(&toBake->sem_bake_done);
    continue;
}

pthread_mutex_unlock(&lock);
}
}

```

Quick checklist for you to apply now

- ☐ Remove or move `if (shutdown_flag) break;` so it only causes exit when queues empty.
- ☐ Make `shutdown_flag` be set **only** after all customers have left (or after queues empty).
- ☐ Ensure `q_push / q_pop` locking policy is consistent with callers (prefer single global `lock` for simplicity).
- ☐ Make sure you `pthread_cond_broadcast(&chef_cv)` whenever you enqueue work (payments/sofa).
- ☐ `pthread_join` chef threads in `main` after shutdown to ensure clean exit.

If you want, paste your `q_push/q_pop` implementations and your `main()` termination code and I'll produce a patched version that compiles and fixes the premature shutdown.

```

#include "shop.h"

// ----- Chef thread -----
void *chef_thread(void *arg) {
    int chef_id = (int)(intptr_t)arg;

```



```

while (!shutdown_flag) {
    Customer *toPay = NULL;
    Customer *toBake = NULL;

    // pick work: payments have priority
    pthread_mutex_lock(&lock);
    while (payment_queue.size == 0 &&
sofa_queue.size == 0) {
        // nothing to do, wait
        if (shutdown_flag) {
            pthread_mutex_unlock(&lock);
            break;
        }
        pthread_cond_wait(&chef_cv, &lock);
    }

    // payment priority
    if (payment_queue.size > 0) {
        toPay = q_pop(&payment_queue);
        // mark that this chef will accept payment for
'toPay'
        pthread_mutex_unlock(&lock);

        // perform accept payment (2 seconds) - atomic
action
        log_chef_action(chef_id, "accepts payment");
        sleep(2);

        // signal customer that payment accepted ->
they can leave
        sem_post(&toPay->sem_payment_ok);

        // after finishing payment, continue loop
        continue;
    }

    // if no payments, check sofa queue to bake
    if (sofa_queue.size > 0) {
        toBake = q_pop(&sofa_queue);
        // Note: sofa seat remains reserved for this
customer until they leave.
    }
}

```

```

        pthread_mutex_unlock(&lock);

        // signal customer to start getcake concurrently
        sem_post(&toBake->sem_served);

        // Chef bakes (2 seconds)
        log_chef_action(chef_id, "bakes");
        sleep(2);

        // baking finished, inform customer that cake is
ready
        sem_post(&toBake->sem_bake_done);

        continue;
    }
    // unlock if fell through
    pthread_mutex_unlock(&lock);
}
return NULL;
}

// ----- Customer thread -----
void *customer_thread(void *arg) {
    Customer *c = (Customer *)arg;

    // enterofficebakery (we simulate action time)
    log_customer_action(c->id, "enters");
    sleep(1); // customer action = 1s

    pthread_mutex_lock(&lock);
    // capacity check done before creating thread — but
double-check here:
    if (current_customers >= MAX_CAPACITY) {
        // bakery full, leaves immediately
        pthread_mutex_unlock(&lock);
        log_customer_action(c->id, "leaves");
        // cleanup
        sem_destroy(&c->sem_served);
        sem_destroy(&c->sem_bake_done);
        sem_destroy(&c->sem_payment_ok);
        free(c);
        return NULL;
    }

```

```

}
// customer is now inside
current_customers++;

// try to sit on sofa immediately if seat free
if (sofa_occupied < SOFA_SEATS) {
    // take seat
    sofa_occupied++;
    pthread_mutex_unlock(&lock);

    // sitOnSofa action (1 sec)
    log_customer_action(c->id, "sits");
    sleep(1);

    // join sofa queue (wait to be served)
    pthread_mutex_lock(&lock);
    q_push(&sofa_queue, c);
    pthread_cond_broadcast(&chef_cv); // signal chefs
that work available
    pthread_mutex_unlock(&lock);
} else {
    // sofa full -> stand
    // join standing_queue and wait for a seat
    q_push(&standing_queue, c);
    log_customer_action(c->id, "stands");
    // wait until signaled that a seat is free and it's our
turn (FIFO)
    while (1) {
        pthread_cond_wait(&standing_cv, &lock);
        // when signaled, check if we are at head of
standing queue and if a sofa seat is available
        if (standing_queue.head == c && sofa_occupied
< SOFA_SEATS) {
            // remove from standing_queue head (we are
head)
            Customer *popped =
q_pop(&standing_queue);
            if (popped != c) {
                // shouldn't happen
                fprintf(stderr, "Error: standing queue
mismatch\n");
            }
        }
    }
}

```

```

        // occupy seat
        sofa_occupied++;
        // release lock and do sit action
        pthread_mutex_unlock(&lock);

        // sitOnSofa action
        log_customer_action(c->id, "sits");
        sleep(1);

        // join sofa queue
        pthread_mutex_lock(&lock);
        q_push(&sofa_queue, c);
        pthread_cond_broadcast(&chef_cv);
        pthread_mutex_unlock(&lock);
        break;
    }
    // otherwise continue waiting
}
}

// now wait to be served: chef will
sem_post(sem_served) to indicate they started serving
(so that getcake runs concurrently)
    sem_wait(&c->sem_served);
    // getcake (1 sec) - must happen concurrently with
    chef's bake
    log_customer_action(c->id, "requests cake");
    sleep(1);

    // wait for chef to complete baking (chef posts
    sem_bake_done after 2s)
    sem_wait(&c->sem_bake_done);

    // after cake finished -> pay (1s)
    log_customer_action(c->id, "pays");
    sleep(1);

    // after pay action is done, we add ourselves to
    payment_queue and wait for an available chef to
    accept payment
    pthread_mutex_lock(&lock);
    q_push(&payment_queue, c);

```

```

        pthread_cond_broadcast(&chef_cv); // notify chefs
        (they prioritize payment)
        pthread_mutex_unlock(&lock);

        // wait until some chef accepts and posts
        sem_payment_ok
        sem_wait(&c->sem_payment_ok);

        // now payment has been accepted by a chef (chef
        spent 2s)
        // customer leaves now; free sofa seat and
        decrement capacity
        pthread_mutex_lock(&lock);
        sofa_occupied--;
        current_customers--;
        // free a sofa seat, so if anybody is standing, wake
        them (the earliest standing customer)
        pthread_cond_broadcast(&standing_cv);
        // also notify chefs in case they were waiting for
        tasks
        pthread_cond_broadcast(&chef_cv);
        pthread_mutex_unlock(&lock);

        log_customer_action(c->id, "leaves");

        // cleanup
        sem_destroy(&c->sem_served);
        sem_destroy(&c->sem_bake_done);
        sem_destroy(&c->sem_payment_ok);
        free(c);
        return NULL;
    }

```

```

0 Customer 1 enters
0 Customer 3 enters
0 Customer 4 enters
0 Customer 2 enters
0 Customer 6 enters
0 Customer 7 enters
0 Customer 9 enters
0 Customer 10 enters
0 Customer 5 enters

```

0 Customer 11 enters
0 Customer 12 enters
0 Customer 8 enters
0 Customer 13 enters
0 Customer 14 enters
0 Customer 15 enters
0 Customer 16 enters
0 Customer 17 enters
0 Customer 18 enters
0 Customer 19 enters
0 Customer 20 enters
0 Customer 21 enters
0 Customer 22 enters
0 Customer 23 enters
0 Customer 24 enters
0 Customer 25 enters
0 Customer 26 enters
1 Customer 1 sits
1 Customer 3 sits
1 1 Customer 4 sits
Customer 9 sits
1 Customer 15 stands
1 Customer 8 stands
1 Customer 2 stands
1 Customer 17 stands
1 Customer 16 stands
1 Customer 5 stands
1 Customer 12 stands
1 Customer 10 stands
1 Customer 6 stands
1 Customer 20 stands
1 Customer 21 stands
1 Customer 19 stands
1 Customer 14 stands
1 Customer 11 stands
1 Customer 24 stands
1 Customer 7 stands
1 Customer 13 stands
1 Customer 18 stands
1 Customer 22 stands
1 Customer 23 stands
1 Customer 26 stands

1 Customer 25 leaves
2 Chef 1 bakes
2 Customer 1 requests cake
2 Customer 3 requests cake
2 Chef 4 bakes
2 Chef 2 bakes
2 Customer 4 requests cake
2 Customer 9 requests cake
2 Chef 3 bakes
4 Customer 1 pays
4 Customer 4 pays
4 Customer 3 pays
4 Customer 9 pays
5 Chef 3 accepts payment
5 Chef 4 accepts payment
5 Chef 1 accepts payment
5 Chef 2 accepts payment
7 Customer 3 leaves
7 Customer 4 leaves
7 Customer 15 sits
7 Customer 9 leaves
7 Customer 8 sits
7 Customer 2 sits
7 Customer 1 leaves
7 Customer 17 sits
8 Chef 4 bakes
8 Customer 15 requests cake
8 Chef 3 bakes
8 Customer 2 requests cake
8 8 Chef 1 bakes
Customer 8 requests cake
8 Chef 2 bakes
8 Customer 17 requests cake
10 Customer 15 pays
10 Customer 8 pays
10 Customer 2 pays
10 Customer 17 pays
11 Chef 3 accepts payment
11 Chef 4 accepts payment
11 Chef 2 accepts payment
11 Chef 1 accepts payment
13 Customer 15 leaves

13 Customer 17 leaves
13 Customer 16 sits
13 Customer 8 leaves
13 Customer 2 leaves
13 Customer 5 sits
14 Chef 4 bakes
14 Customer 16 requests cake
14 Chef 2 bakes
14 Customer 5 requests cake
16 Customer 16 pays
16 Customer 5 pays
17 Chef 2 accepts payment
17 Chef 4 accepts payment
19 Customer 5 leaves
19 Customer 16 leaves
19 Customer 12 sits
20 Chef 1 bakes
20 Customer 12 requests cake
22 Customer 12 pays
23 Chef 2 accepts payment
25 Customer 12 leaves
25 Customer 10 sits
26 Chef 1 bakes
26 Customer 10 requests cake
28 Customer 10 pays
29 Chef 1 accepts payment
31 Customer 10 leaves
31 Customer 6 sits
31 Customer 20 sits

why do 6 and 20 sit but the program ends?

I see what's happening. 👍

The trace shows that after **Customer 10 leaves** at time 31, both **Customer 6** and **Customer 20** manage to sit down — but then the program terminates instead of continuing their lifecycle.

That usually means one of these two:

1. Main thread is not waiting for all customers

- In your `main()`, you likely `pthread_create` a bunch of customer threads, but don't `pthread_join` them all.
- When `main()` finishes (because it thinks all customers are “done”), the process exits, even if Customer 6 and 20 threads are still running.

This exactly matches what you see: new customers sit, but the process ends before they can continue with `requests` `cake` → `pays` → `leaves`.

✓ Fix:

In `main()`, after creating customer threads, call `pthread_join` for each one, or maintain a counter of active customers and join them.

```
c

pthread_t cust_threads[TOTAL_CUSTOMERS];
for (int i = 0; i < TOTAL_CUSTOMERS; i++) {
    Customer *c = make_customer(i+1);
    pthread_create(&cust_threads[i], NULL, customer_thread, c);
}
for (int i = 0; i < TOTAL_CUSTOMERS; i++) {
    pthread_join(cust_threads[i], NULL);
}
```

2. Chef threads exit early

- In your chef code, the `while (!shutdown_flag)` loop depends on `shutdown_flag`.
- If you set `shutdown_flag = 1` once some customers are finished, chefs will stop servicing queues even if others (like 6 and 20) are still waiting.
- Then, those customers can sit but will never get served, and their threads block → program can exit if `main()` doesn't wait.

✓ Fix:

- Don't set `shutdown_flag` until **all customers have left**.
 - Or use a counter of “customers remaining” and only signal shutdown when `remaining == 0`.
-

From your log

- Customers 6 and 20 do sit, so the `standing → sofa` transition works.
- But no chefs bake for them (no `Chef X bakes` after 31). That strongly suggests the **chef threads exited** because `shutdown_flag` was set prematurely, or main terminated.

👉 So the root problem is **program termination / premature shutdown**.

If you want the bakery to keep running until the last customer is served and leaves:

1. Keep a global counter (`remaining_customers`).
2. Only set `shutdown_flag` (and broadcast to chefs) once this counter hits 0.
3. Make sure `main()` joins all customer threads before exiting.

Do you want me to sketch a `main()` structure with proper join + shutdown sequencing so that Customers 6 and 20 (and anyone else) always finish their full flow?

```
#include "shop.h"

// ----- Global bakery state -----
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t chef_cv =
    PTHREAD_COND_INITIALIZER; // chefs wait on this if
                             // nothing to do
pthread_cond_t standing_cv =
    PTHREAD_COND_INITIALIZER; // standing customers
                             // wait for seat
int current_customers = 0; // overall count inside
                           // bakery (max MAX_CAPACITY)
int sofa_occupied = 0; // current occupied sofa seats
                       // (reserved until customer leaves)

Queue sofa_queue; // customers currently on sofa
                  // (FIFO) -> waiting to be served by chefs
Queue standing_queue; // customers standing waiting
                      // for an available sofa
```

```

Queue payment_queue; // customers who have
finished pay action and waiting for chef to accept
payment

// chefs threads will run forever (or until main exits)
int shutdown_flag = 0; // not used for gentle shutdown
in this sample

int main() {
    // initialize queues
    q_init(&sofa_queue);
    q_init(&standing_queue);
    q_init(&payment_queue);

    // start chef threads
    pthread_t chefs[NUM_CHEFS];
    for (int i = 0; i < NUM_CHEFS; ++i) {
        pthread_create(&chefs[i], NULL, chef_thread,
(void*)(intptr_t)(i+1));
    }

    // Read arrival lines until <EOF>
    // Format: "<timestamp> Customer <id>"
    char line[256];
    Arrival *arr_head = NULL, *arr_tail = NULL;
    while (fgets(line, sizeof(line), stdin)) {
        if (strncmp(line, "<EOF>", 5) == 0) break;
        if (strlen(line) == 0) continue;
        int ts, id;
        char word[64];
        if (sscanf(line, "%d %63s %d", &ts, word, &id) == 3)
        {
            if (strcmp(word, "Customer") != 0) {
                fprintf(stderr, "Skipping invalid line: %s", line);
                continue;
            }
            Arrival *a = malloc(sizeof(Arrival));
            a->ts = ts;
            a->id = id;
            a->next = NULL;
            if (!arr_tail) arr_head = arr_tail = a;
            else { arr_tail->next = a; arr_tail = a; }
        }
    }
}

```

```

    } else {
        fprintf(stderr, "Skipping invalid line: %s", line);
    }
}

// schedule customers according to timestamps
// we'll use a simple sleep-until-next-arrival strategy
// program start time is captured implicitly by first
call to program_start_sec()
gettimeofday(&(struct timeval){0}, NULL); // just to
ensure start time established by function

Arrival *cur = arr_head;
long start_time = program_start_sec();
(void)start_time;

while (cur) {
    int target = cur->ts;
    // sleep until target second (relative)
    long now = program_start_sec();
    if (target > now) sleep(target - now);

    // create customer thread if capacity not exceeded
    at creation moment
    pthread_mutex_lock(&lock);
    if (current_customers >= MAX_CAPACITY) {
        // customer finds bakery full and leaves
        immediately (no thread created)
        pthread_mutex_unlock(&lock);
        print_time_pref();
        printf("Customer %d leaves\n", cur->id);
        fflush(stdout);
        Arrival *tmp = cur;
        cur = cur->next;
        free(tmp);
        continue;
    }
    // else create Customer struct and thread
    Customer *c = malloc(sizeof(Customer));
    c->id = cur->id;
    sem_init(&c->sem_served, 0, 0);
    sem_init(&c->sem_bake_done, 0, 0);

```

```

        sem_init(&c->sem_payment_ok, 0, 0);
        c->next = NULL;
        pthread_t tid;
        if (pthread_create(&tid, NULL, customer_thread, c)
!= 0) {
            perror("pthread_create customer");
            exit(1);
        }
        // detach customer thread
        pthread_detach(tid);
        pthread_mutex_unlock(&lock);

        Arrival *tmp = cur;
        cur = cur->next;
        free(tmp);
    }

    // after scheduling all arrivals, wait a little for
    processing to finish
    // (in a real assignment you might join threads or
    have a shutdown protocol)
    // we'll sleep long enough for all work to be done
    (rough heuristic)
    // But avoid indefinite sleep: wait up to a generous
    timeout.
    sleep(30);

    // request shutdown (not strictly necessary)
    pthread_mutex_lock(&lock);
    shutdown_flag = 1;
    pthread_cond_broadcast(&chef_cv);
    pthread_mutex_unlock(&lock);

    for (int i = 0; i < NUM_CHEFS; ++i)
        pthread_join(chefs[i], NULL);

    return 0;
}

```

is this main correct? how do i improve it to wait for all customers to leave before exiting?

Got it 👍 Thanks for pasting your `main()`. Right now, the behavior you saw (ending while customers are still sitting) comes from this part:

```
c

// after scheduling all arrivals, wait a little for processing to finish
sleep(30);
```

That's just a fixed timeout. If some customers are still mid-lifecycle after 30 seconds (like Customer 6 & 20 in your trace), the program shuts down chefs and exits.

Problems in your `main()`

1. Customer threads are detached

```
c

pthread_detach(tid);
```

This means you can't `pthread_join()` them, so `main` has no way of knowing when each customer is really finished.

2. Fixed `sleep(30)`

This is a hack. If the simulation is longer than 30s, you cut customers off. If it's shorter, you waste time.

3. No "all customers left" detection

There's no counter to track how many customers are still inside or active.

How to improve it

Option A – Keep track of customers & join

- Instead of detaching, store customer threads in an array or list.
- Join them at the end.
- That way, `main()` only exits once every customer has finished (`leaves`).

```
c

pthread_t cust_threads[MAX_CUSTOMERS];
int cust_count = 0;
```

...

```
if (pthread_create(&cust_threads[cust_count], NULL, customer_thread, c) != 0) {  
    perror("pthread_create customer");  
    exit(1);  
}  
cust_count++;  
// don't detach
```

At the end:

```
c  
  
for (int i = 0; i < cust_count; i++) {  
    pthread_join(cust_threads[i], NULL);  
}
```

Now `main()` won't exit until all customers are done.

Option B – Use a global counter of “customers remaining”

- Initialize a global `int total_customers` when parsing arrivals.
- Each customer thread, just before exiting, does:

```
c  
  
pthread_mutex_lock(&lock);  
total_customers--;  
if (total_customers == 0) {  
    pthread_cond_broadcast(&all_done_cv);  
}  
pthread_mutex_unlock(&lock);
```

- In `main()`, after creating threads, wait:

```
c  
  
pthread_mutex_lock(&lock);  
while (total_customers > 0) {  
    pthread_cond_wait(&all_done_cv, &lock);  
}  
pthread_mutex_unlock(&lock);
```

This avoids keeping `pthread_t` handles around if you like detaching.

Chef shutdown

Once you know **all customers are done**, you can safely set `shutdown_flag = 1` and broadcast to `chef_cv`. Chefs will exit their loop, and then you can `pthread_join` them cleanly.

✅ So the cleanest fix for you is:

- **Don't detach customer threads.**
 - Keep their `pthread_t`s.
 - `pthread_join` each.
 - Only after that, set `shutdown_flag` and join chefs.
-

Do you want me to show you a fully rewritten `main()` with **customer join** and **proper chef shutdown sequencing** so that no one (like Customer 6 & 20) gets cut off?

yes, edit my main to join them tgt

Perfect 👍 Let's rewrite your `main()` so that:

- We keep track of every customer thread.
- We `pthread_join()` all customers so we know they've **all left the bakery**.
- Only after customers are done, we shut down chefs and `pthread_join()` them too.

Here's your edited `main()`:

```
c

#include "shop.h"

// ----- Global bakery state -----
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t chef_cv = PTHREAD_COND_INITIALIZER; // chefs wait on this if nothing to do
```



```

pthread_cond_t standing_cv = PTHREAD_COND_INITIALIZER; // standing customers wait for
seat
int current_customers = 0; // overall count inside bakery (max MAX_CAPACITY)
int sofa_occupied = 0; // current occupied sofa seats (reserved until customer leaves)

Queue sofa_queue; // customers currently on sofa (FIFO) -> waiting to be served by chefs
Queue standing_queue; // customers standing waiting for an available sofa
Queue payment_queue; // customers who have finished pay action and waiting for chef to
accept payment

int shutdown_flag = 0; // for chef shutdown

int main() {
    // initialize queues
    q_init(&sofa_queue);
    q_init(&standing_queue);
    q_init(&payment_queue);

    // start chef threads
    pthread_t chefs[NUM_CHEFS];
    for (int i = 0; i < NUM_CHEFS; ++i) {
        pthread_create(&chefs[i], NULL, chef_thread, (void*)(intptr_t)(i+1));
    }

    // Read arrival lines until <EOF>
    char line[256];
    Arrival *arr_head = NULL, *arr_tail = NULL;
    int total_customers = 0;
    while (fgets(line, sizeof(line), stdin)) {
        if (strncmp(line, "<EOF>", 5) == 0) break;
        if (strlen(line) == 0) continue;
        int ts, id;
        char word[64];
        if (sscanf(line, "%d %63s %d", &ts, word, &id) == 3) {
            if (strcmp(word, "Customer") != 0) {
                fprintf(stderr, "Skipping invalid line: %s", line);
                continue;
            }
            Arrival *a = malloc(sizeof(Arrival));
            a->ts = ts;
            a->id = id;
            a->next = NULL;

```

```

        if (!arr_tail) arr_head = arr_tail = a;
        else { arr_tail->next = a; arr_tail = a; }
        total_customers++;
    } else {
        fprintf(stderr, "Skipping invalid line: %s", line);
    }
}

// array to store customer thread handles
pthread_t *cust_threads = malloc(sizeof(pthread_t) * total_customers);
int cust_count = 0;

// schedule customers according to timestamps
gettimeofday(&(struct timeval){0}, NULL); // warm-up call

Arrival *cur = arr_head;
long start_time = program_start_sec(); (void)start_time;

while (cur) {
    int target = cur->ts;
    long now = program_start_sec();
    if (target > now) sleep(target - now);

    pthread_mutex_lock(&lock);
    if (current_customers >= MAX_CAPACITY) {
        pthread_mutex_unlock(&lock);
        print_time_pref();
        printf("Customer %d leaves\n", cur->id);
        fflush(stdout);
        Arrival *tmp = cur;
        cur = cur->next;
        free(tmp);
        continue;
    }

    // create Customer struct and thread
    Customer *c = malloc(sizeof(Customer));
    c->id = cur->id;
    sem_init(&c->sem_served, 0, 0);
    sem_init(&c->sem_bake_done, 0, 0);
    sem_init(&c->sem_payment_ok, 0, 0);
    c->next = NULL;

```

```

    if (pthread_create(&cust_threads[cust_count], NULL, customer_thread, c) != 0) {
        perror("pthread_create customer");
        exit(1);
    }
    cust_count++;
    pthread_mutex_unlock(&lock);

    Arrival *tmp = cur;
    cur = cur->next;
    free(tmp);
}

// === Wait for all customer threads ===
for (int i = 0; i < cust_count; i++) {
    pthread_join(cust_threads[i], NULL);
}
free(cust_threads);

// === Once all customers have left, shut down chefs ===
pthread_mutex_lock(&lock);
shutdown_flag = 1;
pthread_cond_broadcast(&chef_cv);
pthread_mutex_unlock(&lock);

for (int i = 0; i < NUM_CHEFS; ++i) {
    pthread_join(chefs[i], NULL);
}

return 0;
}

```

✓ Key Fixes:

- Removed `pthread_detach` (we need to `join` instead).
- Collected all `pthread_t` handles for customers in an array.
- After scheduling, `main()` joins **all customer threads** → guarantees every customer has finished (including `leaves`).
- Then signals shutdown to chefs and joins them.

Now your program will **only exit once every single customer is done and every chef has shut down.**

Do you want me to also add an `all_done_cv` approach (instead of storing pthreads) so you can still detach customers but `main` waits on a condition variable? That would use less memory but more bookkeeping.

```
0 Customer 24 enters
0 Customer 25 enters
0 Customer 26 enters
1 Customer 6 sits
1 Customer 5 sits
```

customer 26 cannot enter as the bakery has reached its limit of 25
customer 26 leaves.
How do i implement that?

```
#include "shop.h"
```

```
// ----- Global bakery state -----
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t chef_cv =
PTHREAD_COND_INITIALIZER; // chefs wait on this if
nothing to do
pthread_cond_t standing_cv =
PTHREAD_COND_INITIALIZER; // standing customers
wait for seat
int current_customers = 0; // overall count inside
bakery (max MAX_CAPACITY)
int sofa_occupied = 0; // current occupied sofa seats
(reserved until customer leaves)
```

```
Queue sofa_queue; // customers currently on sofa
(FIFO) -> waiting to be served by chefs
Queue standing_queue; // customers standing waiting
for an available sofa
Queue payment_queue; // customers who have
finished pay action and waiting for chef to accept
```

payment

int shutdown_flag = 0; // for chef shutdown

```
int main() {
    // initialize queues
    q_init(&sofa_queue);
    q_init(&standing_queue);
    q_init(&payment_queue);

    // start chef threads
    pthread_t chefs[NUM_CHEFS];
    for (int i = 0; i < NUM_CHEFS; ++i) {
        pthread_create(&chefs[i], NULL, chef_thread,
(void*)(intptr_t)(i+1));
    }

    // Read arrival lines until <EOF>
    char line[256];
    Arrival *arr_head = NULL, *arr_tail = NULL;
    int total_customers = 0;
    while (fgets(line, sizeof(line), stdin)) {
        if (strncmp(line, "<EOF>", 5) == 0) break;
        if (strlen(line) == 0) continue;
        int ts, id;
        char word[64];
        if (sscanf(line, "%d %63s %d", &ts, word, &id) == 3)
        {
            if (strcmp(word, "Customer") != 0) {
                fprintf(stderr, "Skipping invalid line: %s", line);
                continue;
            }

            Arrival *a = malloc(sizeof(Arrival));
            a->ts = ts;
            a->id = id;
            a->next = NULL;
            if (!arr_tail) arr_head = arr_tail = a;
            else { arr_tail->next = a; arr_tail = a; }
            total_customers++;
        } else {
            fprintf(stderr, "Skipping invalid line: %s", line);
        }
    }
}
```

```

    }
}

// array to store customer thread handles
pthread_t *cust_threads = malloc(sizeof(pthread_t) *
total_customers);
int cust_count = 0;

// schedule customers according to timestamps
gettimeofday(&(struct timeval){0}, NULL); // warm-up
call

Arrival *cur = arr_head;
long start_time = program_start_sec();
(void)start_time;

while (cur) {
    int target = cur->ts;
    long now = program_start_sec();
    if (target > now) sleep(target - now);

    pthread_mutex_lock(&lock);
    if (current_customers >= MAX_CAPACITY) {
        pthread_mutex_unlock(&lock);
        print_time_pref();
        printf("Customer %d leaves\n", cur->id);
        fflush(stdout);
        Arrival *tmp = cur;
        cur = cur->next;
        free(tmp);
        continue;
    }

    // create Customer struct and thread
    Customer *c = malloc(sizeof(Customer));
    c->id = cur->id;
    sem_init(&c->sem_served, 0, 0);
    sem_init(&c->sem_bake_done, 0, 0);
    sem_init(&c->sem_payment_ok, 0, 0);
    c->next = NULL;

    if (pthread_create(&cust_threads[cust_count],

```

```

NULL, customer_thread, c) != 0) {
    perror("pthread_create customer");
    exit(1);
}
cust_count++;
pthread_mutex_unlock(&lock);

Arrival *tmp = cur;
cur = cur->next;
free(tmp);
}

// === Wait for all customer threads ===
for (int i = 0; i < cust_count; i++) {
    pthread_join(cust_threads[i], NULL);
}
free(cust_threads);

// === Once all customers have left, shut down chefs
===
pthread_mutex_lock(&lock);
shutdown_flag = 1;
pthread_cond_broadcast(&chef_cv);
pthread_mutex_unlock(&lock);

for (int i = 0; i < NUM_CHEFS; ++i) {
    pthread_join(chefs[i], NULL);
}

return 0;
}

// ----- Customer thread -----
void *customer_thread(void *arg) {
    Customer *c = (Customer *)arg;

    // enterofficebakery (we simulate action time)
    log_customer_action(c->id, "enters");
    sleep(1); // customer action = 1s

    pthread_mutex_lock(&lock);
    // capacity check done before creating thread — but

```

double-check here:

```
if (current_customers >= MAX_CAPACITY) {
    // bakery full, leaves immediately
    pthread_mutex_unlock(&lock);
    log_customer_action(c->id, "leaves");
    // cleanup
    sem_destroy(&c->sem_served);
    sem_destroy(&c->sem_bake_done);
    sem_destroy(&c->sem_payment_ok);
    free(c);
    return NULL;
}
// customer is now inside
current_customers++;

// try to sit on sofa immediately if seat free
if (sofa_occupied < SOFA_SEATS) {
    // take seat
    sofa_occupied++;
    pthread_mutex_unlock(&lock);

    // sitOnSofa action (1 sec)
    log_customer_action(c->id, "sits");
    sleep(1);

    // join sofa queue (wait to be served)
    pthread_mutex_lock(&lock);
    q_push(&sofa_queue, c);
    pthread_cond_broadcast(&chef_cv); // signal chefs
that work available
    pthread_mutex_unlock(&lock);
} else {
    // sofa full -> stand
    // join standing_queue and wait for a seat
    q_push(&standing_queue, c);
    log_customer_action(c->id, "stands");
    // wait until signaled that a seat is free and it's our
turn (FIFO)
    while (1) {
        pthread_cond_wait(&standing_cv, &lock);
        // when signaled, check if we are at head of
standing queue and if a sofa seat is available
```



```

        if (standing_queue.head == c && sofa_occupied
< SOFA_SEATS) {
            // remove from standing_queue head (we are
head)
            Customer *popped =
q_pop(&standing_queue);
            if (popped != c) {
                // shouldn't happen
                fprintf(stderr, "Error: standing queue
mismatch\n");
            }
            // occupy seat
            sofa_occupied++;
            // release lock and do sit action
            pthread_mutex_unlock(&lock);

            // sitOnSofa action
            log_customer_action(c->id, "sits");
            sleep(1);

            // join sofa queue
            pthread_mutex_lock(&lock);
            q_push(&sofa_queue, c);
            pthread_cond_broadcast(&chef_cv);
            pthread_mutex_unlock(&lock);
            break;
        }
        // otherwise continue waiting
    }
}

// now wait to be served: chef will
sem_post(sem_served) to indicate they started serving
(so that getcake runs concurrently)
    sem_wait(&c->sem_served);
    // getcake (1 sec) - must happen concurrently with
chef's bake
    log_customer_action(c->id, "requests cake");
    sleep(1);

    // wait for chef to complete baking (chef posts
sem_bake_done after 2s)

```

```

sem_wait(&c->sem_bake_done);

// after cake finished -> pay (1s)
log_customer_action(c->id, "pays");
sleep(1);

// after pay action is done, we add ourselves to
payment_queue and wait for an available chef to
accept payment
pthread_mutex_lock(&lock);
q_push(&payment_queue, c);
pthread_cond_broadcast(&chef_cv); // notify chefs
(they prioritize payment)
pthread_mutex_unlock(&lock);

// wait until some chef accepts and posts
sem_payment_ok
sem_wait(&c->sem_payment_ok);

// now payment has been accepted by a chef (chef
spent 2s)
// customer leaves now; free sofa seat and
decrement capacity
pthread_mutex_lock(&lock);
sofa_occupied--;
current_customers--;
// free a sofa seat, so if anybody is standing, wake
them (the earliest standing customer)
pthread_cond_broadcast(&standing_cv);
// also notify chefs in case they were waiting for
tasks
pthread_cond_broadcast(&chef_cv);
pthread_mutex_unlock(&lock);

log_customer_action(c->id, "leaves");

// cleanup
sem_destroy(&c->sem_served);
sem_destroy(&c->sem_bake_done);
sem_destroy(&c->sem_payment_ok);
free(c);
return NULL;

```

```
}
```

I feel it has already been implemented but the problem persists anyway...

I see what's happening 🧐

You *do* already check the `MAX_CAPACITY` before creating a customer thread in `main` and again at the start of `customer_thread()`. But in your log you got:

```
0 Customer 24 enters
0 Customer 25 enters
0 Customer 26 enters
1 Customer 6 sits
1 Customer 5 sits
```

and then only later:

```
pgsql
```

```
customer 26 cannot enter as the bakery has reached its limit of 25
customer 26 leaves.
```

Why this happens

- In `main()`, you check:

```
c

pthread_mutex_lock(&lock);
if (current_customers >= MAX_CAPACITY) {
    pthread_mutex_unlock(&lock);
    print_time_pref();
    printf("Customer %d leaves\n", cur->id);
    ...
    continue;
}
```

But **you never increment** `current_customers` **there** when you let them in.

Instead, you only increment inside `customer_thread()` → after `"enters"` has

already been printed.

So multiple arrivals at the same timestamp can all pass the check in `main()` before `current_customers` is updated by their threads. This causes “phantom overcapacity” where more than 25 are created, and only inside the customer thread do they discover the bakery is full.

✓ Fix

Increment `current_customers` **immediately in `main()` when admitting** a customer. That way, the admission check is atomic and consistent.

Modify this block in `main()` :

```
c

pthread_mutex_lock(&lock);
if (current_customers >= MAX_CAPACITY) {
    pthread_mutex_unlock(&lock);
    print_time_pref();
    printf("Customer %d leaves\n", cur->id);
    fflush(stdout);
    ...
    continue;
}

// admit customer immediately
current_customers++; // <---- ADD THIS HERE
pthread_mutex_unlock(&lock);
```

Then, inside `customer_thread()`, **remove the second capacity check** (or at least don't increment `current_customers` again). That double-counting is why your log looks inconsistent.

So in `customer_thread()` :

```
c

pthread_mutex_lock(&lock);
// capacity check no longer needed here (already handled in main)
```

```
pthread_mutex_unlock(&lock);
```

Extra polish (for your desired log style)

If you want the message exactly like:

```
pgsql
```

```
customer 26 cannot enter as the bakery has reached its limit of 25
customer 26 leaves.
```

change the log in the `main` capacity rejection case to:

```
c
```

```
print_time_pref();
printf("Customer %d cannot enter as the bakery has reached its limit of %d\n", cur->id,
MAX_CAPACITY);
print_time_pref();
printf("Customer %d leaves\n", cur->id);
fflush(stdout);
```

👉 Do you want me to refactor it so **all admission logic lives only in** `main` (so customer threads always start *inside* the bakery), or keep the “double-check” inside `customer_thread` as a safety net?